

# Functional Requirements (35):

## Authentication & User Management:

1. The system shall allow users to register with a unique username and email with validation (3–20 chars, alphanumeric).
2. The system shall authenticate users via secure login using email and password with JWT token generation.
3. The system shall allow users to log out, invalidating their session token.
4. The system shall provide a "Forgot Password" flow that generates a time-limited reset token (10 min expiry).
5. The system shall allow users to reset their password using a valid reset token.
6. The system shall display a real-time password strength indicator (Very Weak → Very Strong) during registration.
7. The system shall prevent banned users from logging in and display an appropriate error message.

## Player Profile:

8. The system shall display a public gamer profile with bio, avatar, rank badge, preferred games, and playtime stats.
9. The system shall allow users to edit their profile information (avatar, bio, custom status).
10. The system shall allow users to link external gaming accounts (Steam, Discord, Riot Games, Epic Games, Xbox, PlayStation).
11. The system shall allow users to manage notification preferences (email, friend requests, messages, clip likes, team invites).
12. The system shall display a per-game stats dashboard showing K/D ratio, win rate, hours played, kills, deaths, and assists.
13. The system shall allow users to select and edit game-specific combat stats through an interactive game selector.
14. The system shall aggregate individual game stats into a global stats summary on the player profile.

## Game Directory:

15. The system shall display a game catalog with category filters (FPS, MOBA, RPG, Battle Royale).
16. The system shall provide a search bar to filter games by name within the catalog.
17. The system shall display a detailed game page with description, category, cover image, and players who play that game.
18. The system shall support seeding the database with 20+ pre-configured game entries for demonstration.

## AI-Powered Smart Search:

19. The system shall provide an AI chat widget for natural language player search (e.g., "find Gold rank Valorant players").
20. The system shall use a Large Language Model (Groq/Llama 3) to extract structured filters (game, rank, K/D, status) from user queries.
21. The system shall convert AI-extracted filters into MongoDB queries to return matching player results.

## Real-Time Messaging:

22. The system shall allow authenticated users to send direct messages to other players in real-time via Socket.io.
23. The system shall persist all messages in the database with sender, receiver, and timestamp.
24. The system shall display a conversations list showing all active chats with the last message preview.
25. The system shall track and display online/offline/in-game status of users in real-time.

## Friends System:

26. The system shall allow users to send, accept, and decline friend requests.
27. The system shall display a list of the user's current friends.
28. The system shall allow users to unfriend (remove) an existing friend.
29. The system shall provide a user discovery feature to find and browse other players.

## Admin Panel & RBAC:

30. The system shall restrict admin panel access to users with the "admin" role only (Role-Based Access Control).
31. The system shall display a dashboard with platform metrics: total users, active users, and banned users.
32. The system shall allow admins to view all registered users in a paginated, searchable table.
33. The system shall allow admins to ban or unban users, forcing banned users offline immediately.
34. The system shall prevent admins from banning themselves or other admin accounts.
35. The system shall allow admins to change a user's role between "user" and "admin."

## Non-Functional Requirements (25):

### Performance:

1. The system shall load any page within 3 seconds under normal network conditions.
2. The system shall deliver real-time messages with less than 500ms latency using WebSocket connections.
3. The admin user list shall support server-side pagination to handle 100+ users without frontend performance degradation.
4. The AI chat responses shall be returned within 5 seconds under normal API conditions.

### Security:

5. The system shall hash all user passwords using Bcrypt with a salt factor of 10 before storing in the database.
6. The system shall use JWT tokens with a configurable expiry period (default: 7 days) for session management.
7. The system shall implement rate limiting on authentication endpoints (max 20 requests per 15 minutes for registration, 10 for login).
8. The system shall sanitize all database inputs using express-mongo-sanitize to prevent NoSQL injection attacks.
9. The system shall exclude sensitive fields (password, reset tokens) from all API responses using Mongoose select: false.

10. The system shall use CORS restrictions to allow requests only from the configured client URL origin.
11. Password reset tokens shall expire after 10 minutes and be stored as SHA-256 hashes.

## Usability:

12. The system shall provide a responsive UI that works on desktop and mobile screen sizes.
13. The system shall display toast notifications for all user actions (success, error, warning) using react-hot-toast.
14. The system shall provide clear and descriptive error messages for all form validation failures.
15. The system shall use a consistent dark theme ("Emerald Matrix") with glassmorphism design across all pages.
16. The system shall provide intuitive navigation through a responsive Navbar with dropdown menus.

## Maintainability:

17. The system shall follow the MVC (Model-View-Controller) architecture pattern for clear separation of concerns.
18. The system shall use a centralized Axios instance with request/response interceptors for all API communication.
19. The system shall use environment variables (.env) for all configuration secrets (DB URI, JWT secret, API keys).
20. The system shall use a global CSS design system with custom properties (CSS variables) for consistent theming.

## Scalability:

21. The system shall use MongoDB Atlas (cloud-hosted) to support horizontal scaling of the database.
22. The system shall support file uploads (clips, screenshots, avatars) via Multer with organized storage directories.
23. The system shall use Socket.io's event-driven architecture to support concurrent real-time connections.

## Reliability:

24. The system shall handle API failures gracefully by catching errors and returning structured JSON error responses.
25. The system shall automatically redirect users to the login page when their JWT token expires or becomes invalid.